# AI PRACTICE COACH

Act as a Principal Software Architect, Senior Full-Stack Engineer, Product Manager, and AI Systems Designer.

I am a Software Engineer 1 building a portfolio project called **AI Practice Coach**.

The first skill supported will be **Typing**.

The goal is NOT to build a typing website.

The goal is to build an AI-powered coaching platform that learns from a user's performance over time, identifies weaknesses, creates personalized training plans, generates exercises, tracks improvement, and acts as a long-term coach.

Typing is only the first skill module.

Future skill modules may include:

- Coding Practice
- Interview Preparation
- Vocabulary Building
- Language Learning

The architecture must support future skill modules without major refactoring.

---

# PROJECT GOALS

1. Build a real, deployable product.
2. Keep all development and deployment free.
3. Avoid unnecessary complexity.
4. Ship an MVP quickly.
5. Introduce AI only after the foundation exists.
6. Design the system so AI providers can be replaced later.
7. Preserve user history permanently.
8. Build something that demonstrates strong software engineering and practical AI usage.

---

# TECH STACK

Frontend

- React

- Vite
- TypeScript
- TailwindCSS
- React Query
- React Router
- Zustand
- Recharts

Backend

- NestJS
- TypeScript

Database

- MongoDB Atlas Free Tier

Authentication

- JWT

AI

- Groq API

Deployment

Frontend:

- Vercel Free Tier

Backend:

- Render Free Tier

Database:

- MongoDB Atlas Free Tier

Version Control

- GitHub

CI/CD

- GitHub Actions

Containerization

- Docker

---

# IMPORTANT CONSTRAINTS

I have approximately 1 hour per day available.

The architecture must optimize for:

- Simplicity
- Maintainability
- Fast delivery
- Portfolio quality

Do not introduce infrastructure that is not necessary.

Avoid:

- Kafka
- Redis
- Microservices
- Event buses
- Vector databases
- Kubernetes

unless there is a strong justification.

Prefer a modular monolith architecture using NestJS.

---

# LONG-TERM MEMORY REQUIREMENT

The system must preserve user history permanently.

A user may return after:

- 1 day
- 1 week
- 1 month
- 6 months

The AI Coach should remember:

- Historical sessions
- Historical weaknesses
- Historical strengths
- Previous diagnoses

- Previous training plans
- Milestones
- Trends

The user should feel like they have a personal coach rather than a stateless chatbot.

Example:

"Welcome back. Before your break, your biggest challenge was double-letter words. Your accuracy improved from 81% to 90% during your previous training cycle."

---

# PHASE 1 (MVP)

Build a fully functional typing platform before adding AI.

Features:

Authentication

- Register
- Login
- Logout

Typing Engine

- Real-time WPM
- Real-time Accuracy
- Mistyped Word Tracking
- Backspace Tracking
- Session Completion

Dashboard

- Current WPM
- Best WPM
- Average Accuracy
- Session History

Database Persistence

Store every typing session permanently.

Example:

{ "userId": "...", "date": "...", "wpm": 67, "accuracy": 92, "backspaces": 22, "mistakes": [ "receive", "committee" ] }

The MVP should be deployable.

---

# PHASE 2

Learning Profile

Create a permanent Learning Profile.

Example:

{ "userId": "...",

"currentWpm": 67,

"bestWpm": 81,

"averageAccuracy": 91,

"primaryWeaknesses": [],

"strengths": [],

"milestones": [],

"learningStyle": null,

"plateauDetected": false }

The profile should update after every session.

---

# PHASE 3

AI Diagnosis

Integrate Groq.

The AI should analyze:

- Session history
- Mistyped words
- Learning profile

- Performance trends

The AI should identify patterns.

Example:

Mistyped Words:

receive believe piece chief

Diagnosis:

User struggles with IE/EI spelling patterns.

The AI should explain why it reached the conclusion.

Store all diagnoses.

---

# PHASE 4

AI Coach

Build a conversational AI Coach.

The coach should have access to:

- User history
- Learning profile
- Diagnoses
- Milestones
- Recent sessions

The coach should provide:

- Progress analysis
- Recommendations
- Goals
- Personalized advice

The coach should reference historical progress when responding.

---

# PHASE 5

Exercise Generator

Generate personalized typing exercises.

Input:

- Weakness
- Difficulty
- Learning history

Output:

Natural language passages targeting the weakness.

Avoid repetitive word lists.

Store generated exercises.

---

# PHASE 6

Agentic AI

Introduce LangGraph only after Phases 1-5 are complete.

Agents:

1. Observation Agent
2. Memory Agent
3. Diagnosis Agent
4. Planning Agent
5. Content Generation Agent
6. Evaluation Agent

Workflow:

Observation → Memory Retrieval → Diagnosis → Planning → Exercise Generation → Evaluation → Learning Profile Update

The Learning Profile acts as long-term memory.

---

# BACKEND MODULES

Use a modular monolith architecture.

NestJS modules:

auth/ users/ typing/ sessions/ learning-profile/ analytics/ coach/ ai/ agents/ common/

For each module define:

- Controllers
- Services
- DTOs
- Validation
- Mongo Schemas

---

# DATABASE DESIGN

Design collections for:

Users

TypingSessions

LearningProfiles

Diagnoses

TrainingPlans

Milestones

CoachConversations

GeneratedExercises

Include indexes and relationships.

---

# FRONTEND PAGES

Login

Register

Dashboard

Typing Test

Session History

Learning Profile

AI Coach

Settings

---

# DELIVERABLES

Do NOT generate code.

Generate:

1. Complete project architecture
2. Folder structure
3. Database schema design
4. API design
5. NestJS module design
6. React application structure
7. MongoDB indexes
8. Authentication design
9. Deployment architecture
10. Development roadmap
11. Sprint-by-sprint implementation plan
12. Future AI architecture

Act like a Staff Engineer preparing a real project for development.

The design should prioritize shipping a production-quality MVP within 6–8 weeks while keeping the architecture extensible for future AI capabilities.